What Exact Memory Footprints of GPU Operations Save in Copies and in Unified Memory

Nafis Mustakin
University of California, Riverside

October 2026

Abstract

A CUDA program copies its data to the GPU, runs kernels on it, and copies results back. A runtime system that knew exactly which bytes each operation reads and writes, its *footprint*, could leave out of a copy the bytes no later operation reads, and the bytes the GPU has not changed. In graph programs and other irregular code the footprint of a kernel depends on values in memory, so a compiler cannot determine it, and prior systems that use footprints either restrict themselves to regular access patterns or learn them from earlier runs. We measure what exact footprints are worth for copies, using the footprints that the MemPrint hybrid interpreter computes before the program runs on the GPU, and the footprints an NVBit tool records on an NVIDIA A30 as an oracle. We first show that the footprint of one interpreted thread order is not safe to act on. In two GraphBIG programs whose result depends on thread order, the lockstep footprint would leave out bytes that the GPU run needs. We therefore compute the footprint over every thread order the interpreter runs, and count a write as overwriting a copied value only if it happens in every legal order. With these footprints no program loses a needed byte. On 36 small and 37 large benchmarks from Rodinia, GraphBIG and PolyBench-GPU, 22.1% and 4.5% of the bytes copied to the device are never read before they are overwritten. Under a copy cost fitted to the A30, leaving these bytes out speeds up the GPU operations by a geometric mean of 1.018× and 1.056× with the oracle footprints, and saves up to 78.3% of the GPU operation time of `kcore`. With the interpreter's safe footprints, the speedups are 1.008× and 1.052×. The savings come from a few programs, and the gap between the interpreter and the oracle comes from writes that happen only if a loaded value says so, which the interpreter cannot prove happen in every order. We then model the same programs on Unified Memory with less device memory than their data, running each kernel wave by wave of thread blocks, with the costs of page faults and prefetches that we measure on the A30. A prefetch call costs 48 µs, so prefetching every page a wave touches is slower than demand paging (0.929× on the large inputs). Prefetching only runs of contiguous pages long enough to repay the call, and evicting the page used farthest ahead, speeds the 14 large programs that can run in less memory up by a geometric mean of 1.249× over demand paging, with the interpreter's footprints exactly as with the traced ones. On the A30 itself, the interpreter's plans for prefetching before each kernel are identical to the traced ones, and prefetching runs speeds unmodified programs up by 1.057× to 1.134× at half of their memory, across repeated measurements of the same plans. However, these plans do not need exact footprints. Except in the breadth-first searches, the pages a kernel touches in each buffer form one contiguous range, so prefetching the pages between the first and the last that a wave touches in each buffer, which a bounds analysis of the indices or one recorded run could give, gains as much as the footprints in every configuration of the model, and its plans are identical to the traced plans on the A30 in 39 of 42 combinations of program and share of memory.

1 Introduction

GPU programs spend a large share of their time on copies between host and device memory. On the benchmarks of this paper, copies take a median of 23% of the time of all GPU operations on the small inputs and 35% on the large ones. A copy moves a whole buffer, or a range of it, as the programmer wrote it, although the kernels that follow may read only part of what was copied, and a copy back to the host may return bytes that no kernel changed. A runtime system that knew each operation's *footprint*, the bytes it reads and writes, could copy less.

However, the footprint of a kernel in irregular code is chosen by values in memory. A breadth-first search reads the neighbour list of each vertex in its frontier, so the bytes it reads depend on the graph. Static analyses of kernel indices, such as CODA [4] and LADM [3], compute the data each thread block accesses when its indices follow a stride, and CODA falls back to profiling or to interleaving the data across devices when they do not. Runtime systems for Unified Memory, such as SUV [2], decide placement per data structure, and MSched [6] predicts a kernel's working set from its launch arguments, learned from earlier runs. Therefore, it is not known how much exact footprints of irregular kernels are worth.

The MemPrint hybrid interpreter computes exact footprints before the program runs on the GPU. It runs the program's host code natively and interprets each kernel on the real memory contents, so a loaded value, a branch on it and an address computed from it are known exactly. It runs every kernel under four thread orders, because the footprint of a kernel whose threads race through memory can depend on the order. We measure what its footprints save in copies between host and device, against the footprints that an NVBit [7] tool records on the GPU.

Footprints matter more when the program's data does not fit in device memory. With Unified Memory, the GPU moves pages between host and device as kernels touch them, and every page a kernel touches that is not on the device stops the kernel for a page fault. Irregular programs suffer most from such oversubscription [5]. A runtime system that knew which pages each part of a kernel touches could move them before the kernel needs them, and evict the pages that are needed latest. We model this on the same benchmarks, with the footprints of each wave of thread blocks and with fault and prefetch costs measured on the A30, and compare it with demand paging and with prefetching every buffer a kernel uses. We then run the programs on the A30 with Unified Memory and prefetch plans before each kernel. Finally, we ask whether such plans need exact footprints at all, by comparing them with plans that take only the first and last page each wave touches in each buffer, and with plans that repeat the pages of a kernel's previous launch.

In this work, we make the following contributions:

- We show that the footprint of one interpreted thread order is not safe to act on, because in two GraphBIG programs whose result depends on thread order, the lockstep footprint would leave bytes out of a copy that the GPU run needs (Section 3).
- We define the footprint over every interpreted thread order, in which a write overwrites a copied value only if it happens in every legal order, and we compute it both during the interpreter's run and from the references it already records (Section 3).
- We measure, on 36 small and 37 large benchmarks, the bytes of each copy that no operation needs and the GPU time that leaving them out saves, for the traced footprints, the interpreter's footprint over every order, and its lockstep footprint (Section 6.1).
- We model the benchmarks on Unified Memory with less device memory than their data, with kernels run wave by wave of thread blocks, and measure what prefetching and eviction gain

over demand paging with the footprints of each wave, under fault and prefetch costs measured on the A30 (Section 5 and Section 6.2).

- We run 14 unmodified programs on the A30 with Unified Memory and prefetch plans from each footprint, and compare the measured speedups with the model's (Section 6.3).
- We show that for Unified Memory the first and last page that a wave touches in each buffer are enough on these benchmarks, because plans built from them gain as much as plans built from exact footprints, while plans that repeat a kernel's previous launch gain less (Section 6.2 and Section 6.3).

2 Background

Operations and footprints. A CUDA program issues a sequence of *operations* on the GPU: kernel launches, copies between host and device memory, and memory fills. We describe the footprint of an operation in units of 4 bytes of a device allocation: for each unit, whether the operation reads it and whether it writes it. A copy to the device writes the units it copies, and a copy to the host reads them.

The hybrid interpreter. The hybrid interpreter of MemPrint compiles the program's host code against a stand-in CUDA runtime whose device memory is host memory, and runs it natively. When the host code launches a kernel, the interpreter runs all threads of the kernel at once as arrays, on the real contents of memory. It records, for each operation, the units it reads and writes. A reference is *certain* if it happens in the run, and *possible* if it may happen, for example through a pointer the interpreter could not place. Threads of a kernel can race through memory, for example when they lower a value with `atomicMin` or claim a slot of a worklist, and the result can then depend on the order in which the GPU runs them. The interpreter therefore runs each program under four thread orders. The first is *lockstep*, which runs all threads of a kernel together. In the other three, each block runs to completion before the next starts, in the order of block numbers, in reverse order, or in a random order. A unit is written *in every legal order* if a write to it is certain, and its address and the conditions under which it happens depend on no value loaded from memory.

The traced footprints. An NVBit tool records the units each operation of a GPU run reads and writes, on an NVIDIA A30. The traced footprints are exact for that run, and we use them as the oracle a footprint analysis cannot improve on.

3 Footprints over every thread order

The interpreter merges the dependences of its four thread orders, but before this work it wrote out the footprint of the lockstep run only. A runtime system that leaves bytes out of a copy needs a footprint that holds for every order, because the order in which the GPU runs a racing kernel is not known in advance. In `bfs-topo-atomic`, threads lower the level of each vertex with `atomicMin`, and which round lowers a level last depends on the order. Its lockstep footprint would leave out 4 bytes of a copy to the device that the traced run reads. In `connected-comp`, the number of rounds changes from run to run, and its lockstep footprint would leave out 4 bytes of a copy to the host that the traced run wrote.

We define the footprint of an operation over every order as follows. First, we match the operations of each order to those of the lockstep run, from both ends of the sequence, as the interpreter does when it merges dependences. Second, a unit is read or written if some order reads or writes it in the matched operation. Third, a unit is *overwritten* only if every order that issues the operation writes it in every legal order, so that a later read sees this operation's value whatever the order. Fourth, an order that runs rounds the lockstep run does not have adds what those rounds touch to the lockstep run's kernels between the same matched operations, as possible references. The interpreter now writes this footprint during its run. For runs made before, we rebuild it from the per-block references that the interpreter already records for each order. On `gemm` and `bfs-topo-atomic`, the two give identical footprints.

4 What footprints save in copies

We count, for each copy, the units it moves that no operation needs, from a footprint of every operation of the run.

Copies to the device. A unit that a copy to the device writes is *unread* if no later operation reads it before an operation overwrites it. A read of a unit and a write to it in one operation read the old value. A copy back to the host counts as a read, so a unit that the program copies in and straight back out is needed.

Copies to the host. A unit that a copy to the host reads is *unwritten* if no device operation wrote it since it was last copied in either direction, so that the host already has its value. This assumption holds only if the copy returns the unit to the host buffer it came from or went to last, which the footprints do not record, so the unwritten units are an upper bound on what a runtime system could leave out.

Copy cost. We model a copy as a fixed cost plus its bytes over a bandwidth, per direction, from all copies that Nsight Systems profiled in the benchmarks of both input sizes. The fixed cost is the median time of copies of 4 KB or less, and the bandwidth is the median over copies of 1 MB or more. A copy to the device costs 1.0 µs plus 11.9 GB/s, and a copy to the host 1.5 µs plus 9.0 GB/s. A copy that leaves out some units moves each contiguous range of the needed units as a copy of its own, and saves the share of the measured copy time that the model gives, or nothing if the ranges cost more than the whole copy. We report the time saved as a share of the time of all GPU operations of the program, issued one after another.

5 Footprints for Unified Memory

We model each program as if `cudaMallocManaged` allocated its device memory, rather than `cudaMalloc`, so that its device allocations are managed memory. A copy to the device becomes a write by the host, so the pages it writes are on the host afterwards, and a copy to the host becomes a read by the host, which brings back the pages the device holds. Device memory holds a share of the pages that the program's device operations touch, from 25% to 100% in steps of 12.5%, with pages of 64 KB, the basic block in which Unified Memory moves data, or of 4 KB.

Waves of thread blocks. The GPU runs a kernel's blocks as far as they fit at once, so a kernel that touches all of the program's pages may touch only part of them at a time. We run each kernel as a sequence of *waves*. A wave holds as many blocks as the A30 runs at once, in block order, which follows from the threads, registers and shared memory of a block and the 56 streaming multiprocessors of the A30. Each wave takes an equal share of the kernel's profiled time and touches the pages that its blocks touch in the traced run, which the NVBit tool records per block. We simulate a share of memory only if the pages of every wave fit in it, because how a wave thrashes when they do not depends on the order of its blocks' accesses, which the footprints do not record.

Policies. We compare four policies. First, *demand paging* moves a page when a wave touches it and the page is not on the device, and evicts the least recently used page. Second, *buffer prefetching*, as a compiler that knows which data structures a kernel uses could do, moves every page of every buffer the kernel touches before its first wave, as far as they fit beside the wave's own pages, and evicts the least recently used page. Third, *interpreter prefetching* moves, before each wave, the pages that the interpreter's blocks of that wave touch under any of its four thread orders, and evicts the page whose next use in that footprint is farthest away. Fourth, the oracle does the same with the traced footprints. Because a prefetch call has a fixed cost, the interpreter and the oracle also each have a *runs* variant, which prefetches only runs of at least 7 contiguous pages of 64 KB, or 73 of 4 KB, and leaves shorter runs to demand paging. These lengths are the shortest runs for which one prefetch call costs less than faulting on the same pages, at the costs below. Every policy pays for what the traced run actually touches, so a prediction that is wrong costs faults or moves that are not needed.

Plans without the interpreter. We also compare two policies that need no interpreter. First, *range prefetching* moves, before each wave, every page of each buffer from the first page that the wave touches in it to the last, and evicts as interpreter prefetching does. For a kernel whose indices follow a stride, an index analysis such as CODA's [4] gives these ranges before the program runs, and for any kernel, the lowest and highest address that one recorded run touches in each buffer give them for that input. We take them from the traced footprints. Second, *history prefetching* moves the pages that the same wave of the same kernel touched at its previous launch, shifted in each buffer by as many pages as the first page moved between the two launches before, and evicts the least recently used page. A runtime system can record these pages while the program runs, so history prefetching uses nothing but the program's past, and it prefetches nothing at a kernel's first launch. Both policies also have a runs variant.

Costs. We measure the costs of Unified Memory on the A30 with two microbenchmarks. In the first, a kernel touches one word in each block of a managed buffer whose pages are on the host. Faulting on one 64 KB block stalls the kernel for 53 µs, and each further block up to 16 MB adds about 13 µs. With a stride of 4 KB, each further page adds about 1.3 µs. In the second, the host moves blocks to the device with `cudaMemPrefetchAsync`, either in one call or in one call per block. A call costs 48 µs, and the data then moves at 11.2 GB/s. The model charges the faults of a wave the measured stall for that many pages, and a prefetch one call per contiguous run of pages plus the transfer. A managed page is held in one memory at a time, so evicting a page, or touching it from the host, always moves it.

For comparison, we also report the costs that we assumed before measuring them. Under the *assumed costs*, the faults of a wave are handled in batches of up to 256, each of which costs a fixed time of 10, 25 or 50 µs plus the transfer of each page at the bandwidth of the copy model

Footprints	Small inputs (36 programs)					Large inputs (37 programs)				
	HtoD unread	DtoH unwritten	Speedup	≥1% programs	Unsafe	HtoD unread	DtoH unwritten	Speedup	≥1% programs	Unsafe
Traced	22.1%	1.1%	1.018	12	–	4.5%	2.5%	1.056	9	–
Interpreter, all orders	21.9%	1.1%	1.008	8	0	4.4%	2.5%	1.052	4	0
Interpreter, lockstep	22.1%	1.1%	1.017	12	2	4.5%	2.5%	1.055	9	2

Table 1: Copies between host and device with each footprint: the share of the bytes copied to the device that are unread and of the bytes copied to the host that are unwritten, the geometric-mean speedup of all GPU operations from leaving them out, the programs that save at least 1% of their GPU operation time, and the programs on which the footprint marks a unit as not needed that the traced run needs.

of Section 4, a prefetch costs the fixed cost of one copy plus the transfers, and only a page that the device wrote is copied back when it is evicted. We report each policy's speedup over demand paging, as a geometric mean over the programs and shares of memory, under the measured costs unless we say otherwise, and under the assumed costs at 25 µs.

6 Evaluation

Setup. We use the benchmarks, inputs, traces and interpreter runs of MemPrint: 8 programs from Rodinia, 10 from GraphBIG and 20 from PolyBench-GPU, unmodified, on small inputs and on larger ones (GraphBIG on a generated graph of 8,192 vertices). GraphBIG's `betweenness` has no run on the large input. On the small input, PolyBench's `corr` and `covar` launch most of their kernels with no thread block, because the grid size is the input size divided by 256, so their kernels do not run and their copies to the host return what was copied in. We leave both out of the small input. For each program we compare three footprints: the traced footprints, the interpreter's footprint over every order (Section 3), and its lockstep footprint. A footprint is *unsafe* on a program if it marks a unit as not needed that the traced run needs.

6.1 Copies between host and device

Results. Table 1 summarises the results. *With its footprint over every order, the interpreter never leaves out a unit that the GPU run needs, on either input.* Its lockstep footprint is unsafe on 2 programs on each input. The bytes copied to the device that are never read are 22.1% of all such bytes on the small inputs and 4.5% on the large ones, and the interpreter finds almost all of them (21.9% and 4.4%). Leaving out the unneeded units speeds up the GPU operations by a geometric mean of 1.018× and 1.056× with the traced footprints, and by 1.008× and 1.052× with the interpreter's footprint over every order.

Where the savings come from. The savings are concentrated in a few programs (Table 2). In GraphBIG's `kcore`, no operation of the run reads most of the edge array the program copies to the device, so 97% of the bytes copied to the device on the large input are never read, and leaving them out saves 78.3% of its GPU operation time, with the interpreter's footprints as with the traced ones. In PolyBench's `2mm`, `3mm` and `jacobi1d`, the program copies buffers to the device that a kernel overwrites before any kernel reads them. In `jacobi2d` and Rodinia's `nw`, a kernel also overwrites about half of the copied bytes before reading them, but these bytes lie in 998 and 128 separate ranges on the small input, so the fixed cost of copying each needed range between them outweighs the bytes saved, and the model saves nothing.

Program	Suite	Small inputs				Large inputs			
		Copies	Traced	All orders	Lockstep	Copies	Traced	All orders	Lockstep
`kcore`	GraphBIG	68	8.0	5.7	8.0	92	78.3	78.3	78.3
`jacobi1d`	PolyBench	14	0.0	0.0	0.0	57	13.3	13.3	13.3
`3mm`	PolyBench	26	9.0	9.0	9.0	29	11.0	11.0	11.0
`bfs`	Rodinia	36	10.7	1.1	10.7	55	5.4	0.5	5.4
`2mm`	PolyBench	30	5.3	5.3	5.3	35	6.0	6.0	6.0
`gc-topo-thread-centric`	GraphBIG	12	4.7	0.1	4.7	7	1.9	0.0	1.9
`bfs-topo-thread-centric`	GraphBIG	28	4.7	1.2	4.7	24	1.6	0.4	1.6
`connected-comp`	GraphBIG	19	4.5	0.0	3.5	24	2.8	0.0	1.5
`sssp-topo-thread-centric`	GraphBIG	31	3.9	1.0	3.9	35	1.6	0.4	1.6
`backprop`	Rodinia	91	2.8	2.8	2.8	98	0.7	0.7	0.7
`betweenness`	GraphBIG	5	2.2	0.6	2.2	–	–	–	–
`bfs-topo-atomic`	GraphBIG	14	2.1	0.0	1.9	10	0.6	0.0	0.5
`bfs-data-thread-centric`	GraphBIG	22	1.3	1.3	1.3	19	0.6	0.6	0.6

Table 2: The programs whose traced footprints save at least 1% of their GPU operation time on either input: the share of that time spent in copies, and the share saved with each footprint (%).

Model	Inputs	Page	Programs	Shares	Measured costs							Assumed costs	
					Buffers	Interpreter	Traced	Interpreter, runs	Traced, runs	Ranges, runs	History, runs	Buffers	Interpreter
Kernels	Large	64 KB	9 of 37	26	0.348	0.913	0.913	1.120	1.120	1.120	1.067	0.572	1.294
Waves	Large	64 KB	18 of 37	60	0.960	1.002	1.002	1.158	1.158	1.158	1.034	1.023	1.337
Waves	Large	4 KB	14 of 37	55	1.029	0.929	0.929	1.249	1.249	1.329	1.064	1.027	1.407
Waves	Small	64 KB	31 of 36	69	0.861	0.895	0.897	1.073	1.073	1.073	1.005	1.215	1.323
Waves	Small	4 KB	8 of 36	30	0.818	0.675	0.675	1.177	1.177	1.177	1.089	1.221	2.165

Table 3: Unified Memory with each policy: the programs and shares of device memory at which every operation (with kernels as whole operations) or every wave fits in less memory than the program's data, and the geometric-mean speedup over demand paging of buffer prefetching, of prefetching every page with the interpreter's and the traced footprints, of prefetching only long runs of pages with them, and of prefetching only long runs with the ranges of pages and with the history of earlier launches, under the costs measured on the A30, and of buffer and interpreter prefetching under the assumed costs at 25 µs per batch of faults.

The cost of safety. The interpreter's footprint over every order saves less than the traced footprints in the GraphBIG programs and in Rodinia's `bfs` (1.1% against 10.7% on the small input). In these programs a kernel writes a vertex's level or flag only if a loaded value says so, so the interpreter cannot prove that the write happens in every legal order, and it keeps the copied value as possibly read. The lockstep footprint reaches the traced savings on these programs, because the lockstep run does write the units, but the same footprint is unsafe on `bfs-topo-atomic` and `connected-comp`, where the order changes which units a kernel writes.

6.2 Unified Memory with less device memory than data

Results. Table 3 summarises the results. *Under the measured costs, prefetching only long runs of pages with the interpreter's footprints gains as much over demand paging as with the traced footprints*, 1.249× on the large inputs with pages of 4 KB and 1.158× with pages of 64 KB, and on no input or page size does the interpreter fall more than 1% behind the oracle in any configuration. Prefetching every page that a footprint names is slower than demand paging on the large inputs with pages of 4 KB (0.929×, for both footprints), because many waves touch short runs of pages, and a prefetch call for each run costs more than the faults it avoids. Buffer prefetching gains 1.029× on the same configurations. When all of the data fits, prefetching runs still saves part of the faults

of the first touch of each page (1.167× with the interpreter's footprints).

Measured against assumed costs. The assumed costs overstated what footprints gain. Under them, the interpreter's prefetching of every page gains 1.407× on the large inputs with pages of 4 KB, against 1.249× for the best policy under the measured costs. The assumed costs charge a prefetch about 1 µs, where a call costs 48 µs on the A30, and they let the device drop a page it did not write, where the A30 moves every evicted page back to the host.

Where the gain comes from. Prefetching runs gains in two ways. It moves long runs of a wave's pages before the wave starts, so the wave takes fewer faults, and it evicts the page needed latest, so it moves fewer pages: 40% fewer bytes than demand paging over all oversubscribed configurations on the large inputs with pages of 4 KB. Buffer prefetching moves pages that the current wave does not touch, and later waves evict them. In `gramschm`, whose kernels run 768 waves, it moves 96 MB to the device at 75% of the memory where demand paging moves 0.8 MB. Prefetching runs loses to demand paging by more than 1% in 11 of the 55 configurations, all of `3dconv` and `hotspot`, where it prefetches nothing and takes the same faults as demand paging. In these programs, evicting the page used farthest ahead evicts pages that no later kernel uses, and the A30 must move each of them back to the host, while least-recently-used eviction leaves some of them on the device until the program ends.

Waves against whole kernels. With kernels as whole operations, a program can run in less memory only if no single kernel touches all of its pages, and on the large inputs with pages of 64 KB that holds for 9 of the 37 programs. With waves, 18 of them can, among them single-kernel programs such as `2dconv` and `3dconv`. No GraphBIG program can, because GraphBIG launches its kernels with few blocks of 1,024 threads, so each kernel is one wave that touches nearly all of the program's pages.

Plans without the interpreter. *Prefetching runs within the first and last page of each buffer gains as much as prefetching runs with the footprints, in every configuration of the model* (Table 3), and in no configuration does it fall more than 1% behind the traced footprints. Of the 14 programs that we run on the A30, the pages a kernel touches in each buffer form one contiguous range of 64 KB pages in all but `bfs`, where 6% of the pages inside the ranges are not touched. With pages of 4 KB, the ranges of `bfs` and of GraphBIG's `bfs-topo-thread-centric` leave 32% and 32% of their pages untouched. Neither program can run in less memory than its data in any model of the large inputs, however, so the model does not test what these gaps cost. On the large inputs with pages of 4 KB, range prefetching gains more than the footprints (1.329× against 1.249×), in 7 configurations, one of them of `srad` and the rest of `pathfinder`. In `pathfinder`, it gains 1.98× to 2.52× where the footprints gain 1.23× to 1.30×, and both move the same bytes. The pages of one wave leave gaps of 4 KB pages that later waves touch, and the range moves these pages early, in fewer and longer calls, where the footprints leave them to faults. History prefetching gains less, 1.064× on the same configurations and 1.067× with kernels as whole operations, because it prefetches nothing at a kernel's first launch, and programs such as `2dconv`, `gemm` and `hotspot` launch each of their kernels once.

6.3 Unified Memory on the A30

Setup. We run 14 of the large programs on the A30 with Unified Memory, unmodified. We build each program with the shared CUDA runtime and preload a library into it. The library replaces

Program	Memory	Demand (ms)	Speedup on the A30					Model
			Buffers	Interpreter	Traced	Interpreter, runs	Traced, runs	Interpreter, runs
2dconv	50%*	54.0	0.90	1.15	0.96	0.90	0.96	—
3dconv	50%	23.5	0.04	0.85	0.88	1.05	1.04	0.92
3mm	50%	1.6	0.96	1.04	1.08	1.02	1.01	1.04
backprop	50%*	307.9	—	0.94	—	1.07	—	—
bfs	50%*	10.3	1.98	1.86	1.77	1.03	1.02	—
bfs-topo-thread-centric	50%*	4.2	1.22	1.19	1.16	1.04	1.06	—
fdtd-2d	50%*	2.9	0.92	0.96	0.84	1.01	1.07	—
gemm	50%*	4.9	1.21	1.19	1.05	1.17	1.20	—
gramschm	75%	39.1	0.87	0.99	1.01	1.00	1.00	1.00
hotspot	50%*	4.1	1.48	1.44	1.45	1.45	1.48	—
jacobi2d	50%*	554.0	1.94	1.12	1.84	0.90	1.82	—
kcore	50%*	1.4	0.83	0.89	0.81	0.99	1.00	—
pathfinder	50%	37.3	0.60	1.31	1.33	1.32	1.29	1.66
srad	50%*	297.5	1.03	1.00	1.00	1.00	0.99	—

Table 4: The programs on the A30 with Unified Memory, at the smallest share of their 64 KB blocks at which every operation fits (* where none below 100% does): the time with demand paging, each plan's speedup over it, and the speedup that the kernel-level model gives for the interpreter's runs. A dash marks a plan whose runs all failed, or a share the model does not simulate.

`cudaMalloc` with `cudaMallocManaged`, and a copy between host and device with a copy by the host. At the first allocation, it allocates all free device memory except a share of the 64 KB blocks that the program's device operations touch, so that the managed pages have only that share. Before each kernel launch, it prefetches the blocks of a plan with `cudaMemPrefetchAsync`, one call per contiguous run, and after the launch it waits for the kernel. For each kernel, a plan holds the blocks that a policy wants and that the plan believes are not on the device, estimating the driver's eviction as least recently used. The driver decides what to evict, so the plans test prefetching only, and only before a whole kernel, because the host cannot prefetch between the waves of a kernel without changes to the kernel. We run each program at 100%, 75% and 50% of its blocks, 3 times, and report the median of the time spent in kernels, prefetches and copies. The median coefficient of variation over the repetitions is 4%.

Results. *At blocks of 64 KB, the interpreter's plans are identical to the traced plans*, in all 42 of the 42 combinations of program and share, both for prefetching every block and for prefetching runs. On the A30, the two footprints therefore differ only by the variation between runs, which is large under oversubscription. The time of one plan differs between repetitions by up to 10.92×, in `kcore`. At half of the memory, prefetching runs speeds the programs up over demand paging by a geometric mean of 1.057× and 1.134× in the two sets of runs of the same plans, and buffer prefetching by 0.853× (Table 4). At 75%, prefetching runs gains 1.021× and 0.999×, prefetching every block 0.960× and 0.943×, and buffer prefetching 0.774×. When all of the data fits, the geometric mean of every plan is within 4% of demand paging. The gains are concentrated in `pathfinder`, `hotspot` and `gemm`. In `bfs`, prefetching every block gains 1.86× and prefetching runs 1.03×, because its kernels touch short runs that the runs plans leave to faults, which indicates that the cost of a call we measured overstates the cost of prefetching these runs on the A30.

Plans without the interpreter. We run the range and history plans for runs in a third set of runs, together with demand paging and the traced plans for runs, 5 times each. The median coefficient of variation over the repetitions is 6%. At blocks of 64 KB, the range plans are identical to the traced plans in 39 of the 42 combinations of program and share, all but those of `bfs`, so

on the A30 they can differ from the traced plans only in `bfs` and by the variation between runs. That variation is large at half of the memory. Over the 13 programs that ran, the traced plans gain 1.139× and the range plans 1.064×, and most of the difference comes from `jacobi2d`, whose identical plans gain 2.70× and 1.64×. At 75% of the memory the two gain 1.015× and 1.014×. The history plans are identical to the traced plans in 15 combinations, and gain 1.093× at half of the memory and 1.017× at 75%. In `hotspot`, which launches its kernel once, the history plan prefetches nothing and gains 1.01× at half of the memory, where the traced plan gains 1.45×.

Against the model. Most of these shares hold less than one kernel's pages, which the model does not simulate. On the 7 oversubscribed configurations that both cover, the kernel-level model predicts 1.156× for prefetching runs, and the A30 measures 1.097× and 1.089× in the two sets of runs. The model predicts that prefetching every page loses (0.917×) where the A30 gains 1.057×, and both find buffer prefetching far slower than demand paging (0.241× and 0.409×), mostly from `3dconv`, whose buffer plan prefetches 1 GB where the footprints name 16 MB.

Failures. At half of its memory, one kernel of `backprop` touches 140 blocks with room for 74. Of the 756 runs, 9 failed, all of them of `backprop` at this share, with an illegal memory access in the kernel. The failure is not caused by prefetching. When we run the program repeatedly, it fails in every second run under every plan, demand paging included, and it does not fail at 75% or 100% of its memory. In the third set, 10 of the 840 runs failed, all of them of `backprop` at the same share.

7 Related work

Static footprints of thread blocks. CODA [4] inspects the index computations of a kernel and computes the stride between the data of consecutive thread blocks at launch, to place each block's data in the memory of the GPU that runs it. LADM [3] extends this index analysis to hierarchical access patterns and jointly places data and schedules thread blocks on systems of several GPUs. Unlike our footprints, both describe the accesses of a thread block by a stride, and do not determine accesses whose indices are loaded from memory. Our range prefetching (Section 6.2) indicates that for prefetching in Unified Memory, such bounds are as useful as exact footprints on the regular programs of our benchmarks.

Footprints for Unified Memory. SUV [2] uses compiler-inferred access information to place data structures in host or device memory and to prefetch at kernel boundaries, at the granularity of data structures rather than bytes. MSched [6] predicts the pages a kernel will touch from its launch arguments, from the history of earlier runs, and migrates them ahead of time when several programs share a GPU. Our history prefetching predicts from earlier launches within one run, without launch arguments, and therefore has nothing to predict from at a kernel's first launch. ETC [5] manages memory oversubscription without footprints, by overlapping eviction with execution and by reducing thrashing, and finds that irregular programs suffer most from it.

Dependences between operations. Wireframe [1] executes a graph of dependences between thread blocks on the GPU. Footprints of the kind we use here are also what determines the dependences between GPU operations, which MemPrint computes from the same interpreter runs.

8 Limitations

A limitation of the unwritten units is that the footprints do not record which host buffer a copy to the host goes to, which means that the savings on copies to the host assume the copy returns each unit to the buffer it last came from or went to. A limitation of the traced footprints as an oracle is that they come from one GPU run, which means that a footprint can be unsafe for another run of a racing program even where it is safe against this one. A limitation of the footprint we rebuild for runs made before this work is that a copy that only another order's extra rounds issue is not in the per-block references, which means that it is left out of that footprint. The footprint the interpreter now writes during its run includes such a copy. A limitation of the copy model is that it charges each needed range a fixed cost and does not model copies of strided ranges, which means that it may undercount the savings on programs whose unneeded units are scattered.

A limitation of the measured costs is that they come from microbenchmarks that touch or prefetch one buffer in order, which means that they may not hold for the scattered pages of a real kernel, and on `bfs` prefetching short runs gains more on the A30 than the measured cost of a call predicts. A limitation of the hardware runs is that the host can prefetch only before a kernel, which means that the policies that prefetch wave by wave, which gain the most in the model, are not tested on the A30, and testing them would need changes to the kernels or to the driver. A limitation of the plans is that they estimate the driver's eviction as least recently used, which means that a plan may skip blocks the driver has evicted or prefetch blocks still on the device. A limitation of the waves is that they issue a kernel's blocks in fixed groups of equal duration, while the GPU starts a block whenever another finishes, which means that the pages in use at once may differ from a wave's. A limitation of the footprints of blocks is that they do not order the accesses within a block, which means that a program whose kernels each run one wave, such as every GraphBIG program on the large inputs, cannot be modelled in less memory than its data. The model also runs every operation after the previous one, so a prefetch does not overlap the kernel before it, as `cudaMemPrefetchAsync` on another stream could.

A limitation of the range plans is that we take the ranges from the traced footprints rather than from an index analysis or a recorded run, which means that we do not measure what obtaining them costs, and for a kernel whose indices are loaded from memory, ranges recorded on one input may not hold on another. A limitation of the comparison between ranges and footprints is that the only programs whose ranges have gaps, the two breadth-first searches, never run in less memory than their data in the model, which means that our results do not show what exact footprints gain over ranges for irregular programs. We also compare ranges with footprints only for Unified Memory, and not for the copies of Section 6.1, where a range would leave out only the units outside it.

9 Conclusion

Exact footprints of GPU operations show that 22.1% of the bytes the benchmarks copy to the device on the small inputs, and 4.5% on the large ones, are never read, and leaving them out saves up to 78.3% of the GPU operation time of one program but little in most. The interpreter's footprints are safe to act on only when taken over every thread order, at a cost of part of the savings in programs whose writes depend on loaded values. On Unified Memory with less device memory than the program's data, a prefetch call on the A30 costs more than the faults on a few pages, so a footprint pays off only for long runs of contiguous pages. Prefetching such runs wave by wave with the interpreter's footprints gains 1.249× over demand paging in the model, as much as an

oracle, and prefetching them before each kernel gains 1.057× to 1.134× on the A30 at half of the memory, with plans identical to the oracle's, where prefetching whole buffers loses. However, the first and last page that each wave touches in each buffer give the same gains on these benchmarks, so for prefetching in Unified Memory the value of exact footprints remains to be shown on irregular programs that run in less memory than their data. We leave for future work prefetching wave by wave on the GPU, which needs support in the kernel or the driver, ordering the accesses within a block, which programs whose kernels run as one wave would need, and irregular inputs large enough to oversubscribe the device with more than one wave per kernel.

References

[1] AmirAli Abdolrashidi, Devashree Tripathy, Mehmet Esat Belviranli, Laxmi Narayan Bhuyan, and Daniel Wong. Wireframe: Supporting data-dependent parallelism through dependency graph execution in GPUs. In *Proceedings of the 50th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 600–611, 2017.

[2] Pratheek B, Guilherme Cox, Ján Veselý, and Arkaprava Basu. SUV: Static analysis guided unified virtual memory. In *Proceedings of the 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 293–308, 2024.

[3] Mahmoud Khairy, Vadim Nikiforov, David Nellans, and Timothy G. Rogers. Locality-centric data and threadblock management for massive GPUs. In *Proceedings of the 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 1022–1036, 2020.

[4] Hyojong Kim, Ramyad Hadidi, Lifeng Nai, Hyesoon Kim, Nuwan Jayasena, Yasuko Eckert, Onur Kayiran, and Gabriel H. Loh. CODA: Enabling co-location of computation and data for multiple GPU systems. *ACM Transactions on Architecture and Code Optimization*, 15(3):32:1–32:23, 2018.

[5] Chen Li, Rachata Ausavarungnirun, Christopher J. Rossbach, Youtao Zhang, Onur Mutlu, Yang Guo, and Jun Yang. A framework for memory oversubscription management in graphics processing units. In *Proceedings of the 24th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pages 49–63, 2019.

[6] Weihang Shen, Yinqiu Chen, Rong Chen, and Haibo Chen. Towards fully-fledged GPU multitasking via proactive memory scheduling. arXiv:2512.24637, 2025.

[7] Oreste Villa, Mark Stephenson, David Nellans, and Stephen W. Keckler. NVBit: A dynamic binary instrumentation framework for NVIDIA GPUs. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 372–383, 2019.